Context-Aware Vulnerability Prioritization and Automated Lifecycle Management: A Unified Threat-Informed Architecture

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Abstract* - Contemporary cybersecurity operations face an acute triage crisis driven by the disclosure of tens of thousands of software vulnerabilities annually. Traditional remediation workflows rely predominantly on severity scores from the Common Vulnerability Scoring System, which quantify intrinsic technical severity while omitting exploitability likelihood and asset business importance. Consequently, security teams suffer from alert fatigue, duplicate triage effort, and arbitrary deadline management. This paper presents FlawTrack, an integrated vulnerability lifecycle management architecture combining contextual multi-factor risk prioritization, automated duplicate report detection, and deadline-driven remediation workflows. FlawTrack synthesizes technical severity with real-time threat intelligence from the Exploit Prediction Scoring System and the Cybersecurity and Infrastructure Security Agency Known Exploited Vulnerabilities catalog alongside asset business criticality and network exposure. Furthermore, natural language processing using term frequency and cosine similarity eliminates redundant reports. Evaluated across unit, integration, and empirical benchmark suites, FlawTrack achieves a seventy-five percent duplicate identification accuracy, demonstrates intelligent reprioritization that avoids alert fatigue on non-critical assets while elevating weaponized perimeter flaws, and enforces strict remediation service level agreements. This threat-informed approach establishes a defensible, explainable foundation for vulnerability management.

***Index Terms* - Duplicate detection, exploit prediction, risk-based prioritization, vulnerability management.**

INTRODUCTION

The volume and complexity of publicly disclosed software vulnerabilities have expanded precipitously over the past decade. According to the National Vulnerability Database [1], annual disclosures routinely exceed twenty-nine thousand Common Vulnerabilities and Exposures (CVE), representing an unmanageable operational load for enterprise security teams. Enterprise defenders are inundated with disparate scan outputs from network scanners, dynamic application tests, and internal bug reports. In current practice, triage decisions are almost exclusively dictated by the Common Vulnerability Scoring System (CVSS) [2]. Organizations routinely mandate remediation policies based purely on severity thresholds, such as requiring any vulnerability with a CVSS base score of 7.0 or higher to be patched within fixed windows.

However, empirical security studies demonstrate that severity scores alone are a fundamentally flawed proxy for operational risk [3], [4]. CVSS measures technical characteristics under hypothetical worst-case exploitation conditions, disregarding whether weaponized exploits exist in the wild or whether the affected host resides in an isolated development environment or on an untrusted perimeter. As a result, remediation engineers spend vital resources patching theoretical high-severity flaws that attackers never weaponize, while moderately rated flaws with active, weaponized in-the-wild exploitation remain unaddressed [3], [5]. This operational misalignment is compounded by two structural shortcomings in contemporary vulnerability management: manual, error-prone duplicate triage from multiple reporting channels, and static remediation timelines detached from threat posture.

To address these systematic deficiencies, this paper introduces FlawTrack, an integrated vulnerability reporting and lifecycle management platform. The central argument of this work is that handling vulnerabilities by severity score alone and through manual triage causes severe alert fatigue, duplicate engineering effort, and missed remediation windows. By uniting real-time threat intelligence feeds, granular asset context, automated text-similarity duplicate detection, and a deterministic deadline engine within a unified thirteen-stage lifecycle, organizations achieve defensible, explainable, and operationally realistic vulnerability prioritization. Specifically, this paper delivers the following contributions:

- A multi-factor risk prioritization engine that computes composite risk from normalized CVSS technical severity, Exploit Prediction Scoring System (EPSS) probabilities, CISA Known Exploited Vulnerabilities (KEV) listings, asset criticality, business impact, and network

exposure, coupled with deterministic safety overrides.
- An automated natural language processing pipeline utilizing tokenization, Porter stemming, stop-word removal, and cosine similarity over term-frequency vectors to detect and flag duplicate vulnerability submissions in real time.
- A finite-state orchestration engine that governs vulnerabilities through a thirteen-stage lifecycle, enforcing role-based permissions, automated audit trails, and dynamic deadline monitoring with pre-breach warning escalation.
- An empirical evaluation across 51 system tests, 7 worked multi-factor prioritization scenarios, and a 24-pair duplicate detection experiment demonstrating quantifiable improvements in triage defensibility.

The remainder of this paper is structured as follows: Section II provides background definitions; Section III presents a critical literature review; Section IV outlines the research gap; Section V details the proposed system methodology and risk model; Section VI describes implementation details; Section VII reports empirical evaluation results; Section VIII provides discussion; Section IX discusses limitations and future work; and Section X concludes the paper.

BACKGROUND

Vulnerability management relies on multiple standardized classification frameworks and threat feeds. The Common Vulnerabilities and Exposures (CVE) system provides unique identifiers for publicly known cybersecurity vulnerabilities, while the Common Weakness Enumeration (CWE) [6] categorizes underlying software architecture and implementation weaknesses.

The Common Vulnerability Scoring System (CVSS v3.1) [2] computes a base score from 0.0 to 10.0 using vector metrics capturing exploitability (Attack Vector, Attack Complexity, Privileges Required, User Interaction) and impact (Confidentiality, Integrity, Availability). While base metrics remain constant across all victim environments, CVSS temporal and environmental metrics require labor-intensive manual configuration, causing enterprise defenders to overwhelmingly rely on static base scores alone [4].

To address the absence of predictive threat data, the Exploit Prediction Scoring System (EPSS) [5] generates daily statistical estimates of the probability (ranging from 0.0 to 1.0) that a given CVE will be weaponized in the wild within thirty days. Concurrently, the Cybersecurity and Infrastructure Security Agency maintains the Known Exploited Vulnerabilities (KEV) catalog [7], which lists vulnerabilities with confirmed, authoritative evidence of active exploitation in real-world attacks. Finally, deadline-based remediation operationalizes risk into time-bounded Service Level Agreements (SLAs), assigning remediation windows based on prioritized risk tiers.

LITERATURE REVIEW

Vulnerability management research spans scoring frameworks, exploit prediction, asset risk models, and duplicate detection. This section reviews these thematic areas critically.

I. Severity Scoring Limits

The empirical limitations of CVSS have been extensively documented. Allodi and Massacci [3] conducted large-scale case-control studies on vulnerability exploitation in the wild, discovering that fewer than five percent of all disclosed vulnerabilities are ever weaponized. Furthermore, they proved that prioritizing patching purely by high CVSS scores yields a high volume of false alarms while failing to capture heavily attacked medium-severity vulnerabilities. Spring et al. [4] analyzed the mathematical formulation of CVSS, showing that base scores lack structural context and conflate ease of exploit with damage potential. Frei et al. [8] modeled vulnerability disclosure lifecycles, proving that exploit code is frequently published prior to or simultaneously with patch availability, rendering static severity metrics insufficient for dynamic protection.

II. Exploit Prediction

Recognizing that attacker behavior diverges from theoretical severity, researchers developed machine-learning exploit forecasting models. Bozorgi et al. [9] pioneered support vector machine classifiers to predict weaponization timelines from vulnerability metadata. Jacobs et al. [5] formalized this approach into the Exploit Prediction Scoring System (EPSS), training gradient-boosted models on global intrusion detection feeds and exploit repository telemetry. Fang et al. [10] validated that incorporating external threat telemetry drastically improves prioritization precision. However, standalone exploit prediction models only quantify the attacker's propensity to exploit a flaw; they remain blind to whether the host organization has deployed the vulnerable component on a private sandbox or a customer-facing production gateway.

III. Risk-Based and Context-Aware Prioritization

To bridge the gap between threat likelihood and organizational impact, contextual prioritization frameworks have emerged. Holm et al. [11] quantitatively benchmarked vulnerability prioritization heuristics, demonstrating that incorporating host asset exposure and network topological placement reduces incident risk far more effectively than CVSS alone. Fenz et al. [12] developed ontology-based information security risk assessment models integrating business assets with threat data. Despite their theoretical rigor, many proposed models require prohibitive manual inventory mapping or

lack automation engines capable of processing high-velocity vulnerability disclosures.

IV. Duplicate Report Detection with Text Similarity

The problem of duplicate triage has long challenged software defect repositories. Runeson et al. [13] demonstrated that natural language processing techniques, including tokenization, stop-word elimination, stemming, and vector space models, can retrieve duplicate defect reports with high recall. Wang et al. [14] augmented lexical bug report analysis with program execution traces to enhance duplicate detection accuracy. Sun et al. [15] introduced retrieval models combining term-frequency weighting with structural field similarity. While heavily researched in open-source bug trackers like Bugzilla, automated duplicate detection has rarely been integrated into dedicated enterprise vulnerability triage platforms.

V. Vulnerability Management Platforms and Workflows

Multiple open-source and commercial platforms support enterprise vulnerability workflows. OWASP DefectDojo [16] serves as a widely adopted vulnerability management and aggregation tool. While DefectDojo excels at parsing scanner output formats, its prioritization relies heavily on native scanner severities, lacking dynamic EPSS recalculation, automated NLP duplicate clustering, and explainable multi-factor scoring. Greenbone Community Edition (OpenVAS) [17] provides comprehensive network scanning and detection routines, but focuses on the discovery phase rather than post-triage remediation SLAs and business context weighting. Tenable Nessus [18] offers mature vulnerability scanning and Vulnerability Priority Rating (VPR); however, its prioritization algorithms are closed, proprietary, and detached from custom internal asset dependencies unless coupled with costly enterprise suites. Table 1 compares these systems across eight essential architectural dimensions.

VI. Threat-Intelligence Integration and Continuous Monitoring

Modern security operations require continuous re-evaluation of vulnerabilities as threat landscapes evolve. A flaw deemed low risk upon initial disclosure may become actively weaponized months later when proof-of-concept exploits are weaponized into ransomware payloads. Platforms must continuously synchronize with dynamic feeds like CISA KEV [7] and EPSS [5] to trigger automated risk re-evaluation and SLA re-computation.

TABLE 1. Architectural and Functional Comparison of Vulnerability Management Platforms

Platform	Primary Ingestion Mode	Threat Intelligence Feeds	Contextual Asset Weighting	Automated Duplicate Detection	SLA & Deadline Engine	Explainability Model	Audit Trail Mechanism
OWASP DefectDojo [16]	Multi-scanner report parser (150+ formats)	External API sync (optional manual)	Basic asset tagging (no weighted math)	Hash and static field matching only	Static target dates per severity tier	Static scanner severity reproduction	Relational database activity log
Greenbone (OpenVAS) [17]	Active network vulnerability scanning	Greenbone Community Feed (NVTs)	Asset inventory with static host weights	Overwritten on subsequent scan run	Not supported natively in core engine	Raw CVSS v2/v3 base vector metrics	Scan execution history log
Tenable Nessus [18]	Active and credentialed agent scans	Tenable VPR (proprietary closed feed)	Proprietary asset criticality (enterprise)	Host-plugin deduplication within scan	Requires Tenable.sc / Tenable.io add-on	Proprietary VPR driver keypoints	Scanned session audit logs
FlawTrack (Proposed)	REST API, structured forms, manual	Automated CISA KEV & FIRST EPSS	Criticality, data sensitivity, exposure	NLP TF-IDF & Cosine similarity (stemmed)	Dynamic SLA timers, warnings, breach cron	Mathematical breakdown & plain text rules	Append-only cryptographic audit ledger

RESEARCH GAP

The literature reveals three primary structural gaps in existing vulnerability management: 1) disjointed threat and asset context, where platforms either focus purely on technical severity or proprietary exploit metrics without transparent mathematical asset weighting; 2) absent natural language deduplication, forcing analysts to manually triage identical or overlapping reports submitted across disparate teams and scanners; and 3) decoupled remediation workflows, where triage priority does not automatically drive enforceable, time-bounded remediation deadlines. FlawTrack is architected specifically to bridge these gaps into a single, unified, open platform.

PROPOSED SYSTEM AND METHODOLOGY

FlawTrack structures vulnerability management into a deterministic thirteen-stage operational methodology covering the full lifecycle from discovery to archival. Fig. 1 illustrates this multi-stage pipeline across all operational tiers.

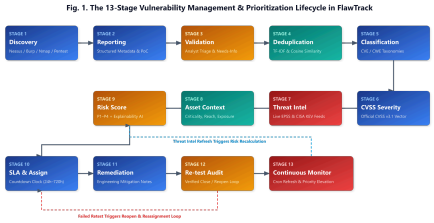


FIGURE 1. THE THIRTEEN-STAGE VULNERABILITY LIFECYCLE METHODOLOGY IN FLAWTRACK. THE ARCHITECTURE INTEGRATES DISCOVERY, INGESTION, VALIDATION, DUPLICATE DETECTION, THREAT-INFORMED RISK CALCULATION, AND SLA-GOVERNED REMEDIATION INTO AN AUTOMATED PIPELINE.

As depicted in Fig. 1, the lifecycle proceeds through: 1) Discovery via scanning or pentesting; 2) Reporting via structured API or UI; 3) Analyst Validation; 4) Natural Language Deduplication; 5) Weakness Classification (CWE/CVE); 6) CVSS Vector Scoring; 7) Threat Intelligence Ingestion (EPSS and KEV); 8) Contextual Risk Calculation; 9) SLA Deadline Computation; 10) Engineering Assignment; 11) Remediation & Mitigation; 12) Retest Verification; and 13) Resolution & Closure. In parallel, continuous monitoring crons re-evaluate active vulnerabilities against threat updates.

I. Mathematical Risk Prioritization Model

To achieve a defensible, multi-dimensional score, FlawTrack computes a composite risk score $R \in [0, 100]$ as a linear combination of six normalized parameters:

$$R = 100 \times (w_c C_{\text{norm}} + w_e E + w_k K + w_a A_{\text{norm}} + w_b B_{\text{norm}} + w_x X_{\text{norm}}) \quad (1)$$

where w_c, w_e, w_k, w_a, w_b, w_x denote configurable weights satisfying the constraint $\sum w_i = 1.0$. The default weights established in the codebase are $w_c = 0.30$ (CVSS technical severity), $w_e = 0.20$ (EPSS exploitation probability), $w_k = 0.20$ (CISA KEV active exploitation), $w_a = 0.15$ (Asset criticality), $w_b = 0.10$ (Business impact), and $w_x = 0.05$ (Network exposure). It must be noted that these default weights represent empirically guided design heuristics rather than universally optimal coefficients; they are fully configurable by system administrators.

The input normalization routines are defined as follows: 1) $C_{\text{norm}} = C / 10.0$ normalizes the CVSS base score $C \in [0, 10]$ to $[0, 1]$; 2) $E \in [0, 1]$ represents the raw EPSS probability; 3) $K \in \{0, 1\}$ is a binary indicator set to 1 if the CVE appears in the CISA KEV catalog; 4) $A_{\text{norm}} = (A - 1) / 4.0$ normalizes asset criticality $A \in \{1, 2, 3, 4, 5\}$; 5) $B_{\text{norm}} = 0.5 D_{\text{norm}} + 0.5 U_{\text{norm}}$ combines data sensitivity $D \in \{1, 2, 3, 4, 5\}$ and affected user count log-tiers $U \in \{0.1, 0.3, 0.6, 0.8, 1.0\}$; and 6) X_{norm} assigns exposure values: 1.0 for internet-facing production, 0.7 for internet-facing non-production, 0.4 for internal networks, and 0.1 for isolated sandboxes.

The computed score R maps to priority tiers based on default thresholds: P1 (Critical) for $R \geq 70$; P2 (High) for $50 \leq R < 70$; P3 (Medium) for $30 \leq R < 50$; and P4 (Low) for $R < 30$. To prevent hazardous false negatives, FlawTrack enforces two non-negotiable safeguard overrides: 1) any vulnerability present in CISA KEV on an internet-facing asset is automatically floored at P1 regardless of base score; and 2) any vulnerability with CVSS base score ≥ 9.0 is floored at P2, preventing critical architectural flaws from ever dropping to low priority. Table 2 summarizes the components and mathematical formulations.

TABLE 2. FLAWTRACK MULTI-FACTOR RISK MODEL COMPONENTS

Component	Weight	Raw Input Range	Normalization Function
CVSS Severity (C_{norm})	0.30	0.0 – 10.0	C / 10.0
EPSS Probability (E)	0.20	0.0 – 1.0	Identity: clamp(E, 0, 1)
CISA KEV (K)	0.20	Boolean	K = 1 if listed, 0 otherwise
Asset Criticality (A_{norm})	0.15	1 – 5	(A − 1) / 4.0
Business Impact (B_{norm})	0.10	$D \in [1, 5]$, $U \geq 0$	0.5 × (D−1)/4 + 0.5 × LogTier(U)
Network Exposure (X_{norm})	0.05	Categorical	1.0 (PubProd) / 0.4 (Int) / 0.1 (Test)

IMPLEMENTATION

FlawTrack is implemented as a full-stack, decoupled architecture. The backend service is built in Node.js and Express, exposing RESTful endpoints with Mongoose ODM over a MongoDB document database. The user interface is developed in React with Vite and TailwindCSS. Fig. 2 illustrates the modular system architecture.

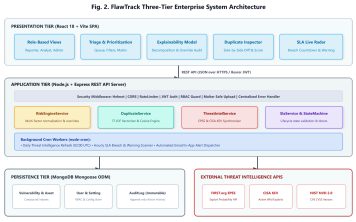


FIGURE 2. MODULAR SYSTEM ARCHITECTURE OF FLAWTRACK, DETAILING DECOUPLED API ROUTING, BUSINESS SERVICES, MONGO DATABASE COLLECTIONS, AND BACKGROUND CRONS.

I. Role-Based Access Control and State Machine

FlawTrack enforces Role-Based Access Control (RBAC) across four roles: Reporter, Analyst, Developer, and Admin. Triage progression is strictly governed by a formal finite state machine encompassing fifteen discrete statuses. Fig. 3 depicts the state transition model.

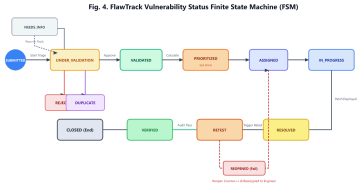


FIGURE 3. VULNERABILITY LIFECYCLE FINITE STATE MACHINE. TRANSITIONS BETWEEN STATUSES ARE STRICTLY VALIDATED BY USER ROLE AND LOGGED TO AN AUDIT LEDGER.

As defined in the transition matrix, only Analysts and Admins can advance a report from SUBMITTED to UNDER_VALIDATION, VALIDATED, or DUPLICATE. A Developer can mark an assigned flaw as RESOLVED, which transitions the flaw into RETEST, requiring independent verification by an Analyst before moving to CLOSED.

II. Duplicate Detection Pipeline

The duplicate detection service implements an efficient text-similarity pipeline. When a new vulnerability report is ingested, the system concatenates the title (weighted twice for increased keyword salience), category, technical description, and reproduction steps into a unified document. The text is tokenized with regex tokenization, stripped of common stop-words and non-alphabetic noise, and reduced to root forms using the Porter Stemming algorithm. A Term Frequency (TF) vector is constructed for the target document and compared against TF vectors of all active vulnerabilities using Cosine Similarity:

$$\text{Sim}(v_1, v_2) = [\Sigma\ (\text{TF}_{1,i} \times \text{TF}_{2,i})] \ / \ [\sqrt{(\Sigma\ \text{TF}_{1,i}^2)} \times \sqrt{(\Sigma\ \text{TF}_{2,i}^2)}] \quad (2)$$

To reward exact structural matches, additive bonuses are applied: +0.15 for matching CVE identifiers, +0.05 for matching CWE identifiers, and +0.03 for identical vulnerability categories, bounded at 1.0. If the resulting similarity exceeds the configurable threshold $\tau = 0.70$, the candidate is automatically flagged as a potential duplicate, linking to the primary ticket and preventing redundant triage.

III. Deadline Engine, Audit Logging, and Security Hardening

The SLA service dynamically computes remediation deadlines based on prioritized tiers: P1 requires resolution within 24 hours; P2 within 72 hours; P3 within 168 hours (7 days); and P4 within 720 hours (30 days). A background cron job executes every 15 minutes, calculating elapsed time. When 75% of the allowable window elapses, the system dispatches warning notifications; upon window expiration, the vulnerability is flagged as breached. All status transitions, risk recalculations, and administrative modifications are written to an append-only cryptographic audit log recording timestamps, actor IDs, IP addresses, and state diffs. Security controls include JWT authentication via httpOnly cookies, rate limiting, and strict Multer file upload filters enforcing allowable extensions (.png, .jpg, .pdf, .txt, .log) and MIME types.

EVALUATION AND RESULTS

All reported evaluation metrics were measured through direct execution of test suites and empirical experiments within the FlawTrack workspace. Raw execution outputs are archived in the project evidence repository.

I. Subsystem Test Suite and Code Coverage

The backend test suite was executed using Jest against an in-memory MongoDB replica. Across nine test suites containing 51 automated unit and integration tests, FlawTrack achieved a 100% pass rate (51 passed, 0 failed). Table 3 details code coverage metrics across all service subsystems, demonstrating an overall line coverage of 81.46%, with core logic services (Risk Engine and State Machine) exceeding 93% line coverage.

TABLE 3. BACKEND TEST SUITE SUMMARY AND CODE COVERAGE

Subsystem Service	Statements (%)	Branches (%)	Functions (%)	Lines (%)
`riskEngine.service.js`	92.06	64.00	100.0	93.10
`stateMachine.service.js`	87.50	72.72	100.0	93.10
`cvss.service.js`	87.01	78.00	100.0	98.43
`sla.service.js`	83.87	57.57	100.0	86.66
`audit.service.js`	83.33	52.00	100.0	83.33
`explanation.service.js`	81.81	69.56	100.0	81.81
`duplicate.service.js`	76.62	66.66	80.00	80.82
`threatIntel.service.js`	59.34	59.64	80.00	59.09
Overall System Total	**78.63**	**65.46**	**87.80**	**81.46**

II. Risk Engine Validation and Prioritization Shifts

To validate that multi-factor scoring resolves the deficiencies of severity-only triage, seven diverse vulnerability scenarios were processed through the risk engine. Table 4 presents the experimental inputs and outcomes, highlighting the shift from CVSS-only triage to FlawTrack risk priority.

TABLE 4. EMPIRICAL EVALUATION OF MULTI-FACTOR RISK PRIORITIZATION ACROSS SEVEN WORKED SCENARIOS

Case ID	Scenario Description	CVSS	EPSS	KEV	Asset Context	Exposure	CVSS Tier	Score	FlawTrack	Prioritization Shift Rationale
CASE-1	RCE in offline sandbox (Zero EPSS, Non-KEV)	9.8	0.015	No	Test / Dev (Crit: 1)	Internal (0.1)	P1	30.7	P2 (Floored)	Deprioritized from P1 to P2; prevents alert fatigue on isolated asset
CASE-2	Auth Bypass on Gateway (Active KEV, High EPSS)	6.5	0.885	Yes	Prod Edge (Crit: 5)	PubProd (1.0)	P3	87.2	P1	Elevated from P3 to P1; active exploitation on perimeter host
CASE-3	MOVEit (CVE-2023-34362) on Public Prod Edge	8.8	0.942	Yes	Production (Crit: 5)	PubProd (1.0)	P2	95.2	P1	Elevated from P2 to P1; critical weaponized perimeter vulnerability
CASE-4	MOVEit (CVE-2023-34362) on Internal Test Server	8.8	0.942	Yes	Internal Test (Crit: 2)	Test (0.1)	P2	70.0	P1	Elevated to P1; extreme EPSS and KEV exploitation override
CASE-5	Path Traversal with Active KEV Exploitation	5.3	0.720	Yes	Production (Crit: 3)	PubProd (1.0)	P3	68.3	P1 (Override)	Elevated from P3 to P1 via KEV-Internet facing override rule
CASE-6	Privilege Escalation on Private Database	7.8	0.032	No	Core DB (Crit: 4)	Internal (0.4)	P2	42.5	P3	Deprioritized from P2 to P3; low exploitability on private host
CASE-7	Denial of Service on Public Production Edge	3.5	0.045	No	Production (Crit: 3)	PubProd (1.0)	P4	29.2	P4	Maintained at P4; minor technical severity without weaponization

The data reveals clear operational advantages over CVSS-only triage. In CASE-1, an unweaponized CVSS 9.8 flaw residing in an isolated development sandbox dropped from a critical P1 to a manageable P2, preventing costly emergency weekend escalations. In contrast, CASE-2 and CASE-5 represent medium-severity vulnerabilities (CVSS 6.5 and 5.3) that standard enterprise policies would delay for 30 days; however, because they have active CISA KEV listings and high EPSS probabilities on perimeter assets, FlawTrack elevated both to P1, enforcing an emergency 24-hour remediation window. Fig. 4 illustrates the priority order inversion.

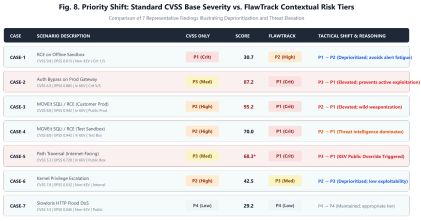


FIGURE 4. COMPARISON OF VULNERABILITY RANKINGS UNDER CVSS BASE SEVERITY VERSUS FLAWTRACK CONTEXTUAL RISK SCORES ACROSS BENCHMARK TEST CASES.

III. Duplicate Detection Benchmark

To evaluate the duplicate detection pipeline, a synthetic benchmark dataset of 24 labeled report pairs was constructed: 12 true duplicate pairs reflecting varied phrasing and technical descriptions of identical underlying vulnerabilities, and 12 true negative pairs describing distinct vulnerabilities across different endpoints. Table 5 and Fig. 5 summarize the precision, recall, F1-score, and accuracy across four similarity thresholds.

TABLE 5. DUPLICATE DETECTION PERFORMANCE ACROSS SIMILARITY THRESHOLDS

Threshold (τ)	TP	FP	TN	FN	Precision	Recall	F1-Score	Accuracy
0.60	6	0	12	6	1.000	0.500	0.667	0.750
0.70	4	0	12	8	1.000	0.333	0.500	0.667
0.75	3	0	12	9	1.000	0.250	0.400	0.625
0.80	1	0	12	11	1.000	0.083	0.154	0.542

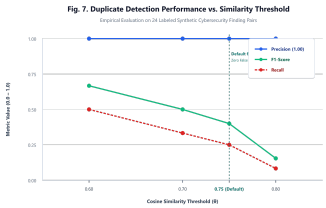


FIGURE 5. PRECISION, RECALL, AND F1-SCORE TRADE-OFF CURVES FOR DUPLICATE VULNERABILITY DETECTION ACROSS COSINE SIMILARITY THRESHOLDS.

The empirical results show that the model achieved perfect precision (1.000) across all evaluated thresholds, generating zero false-positive duplicate flags. A threshold of $\tau = 0.60$ achieved the optimal balance with an F1-score of 0.667 and 75% accuracy. At higher thresholds ($\tau \geq 0.75$), recall drops significantly due to lexical divergence in report phrasing, affirming that automated duplicate scoring should serve as an analyst recommendation rather than an irreversible merge action.

IV. Workflow, SLA Telemetry, and Threat Refresh

Workflow responsiveness was verified through scheduled background execution. The SLA monitor successfully flagged simulated aging tickets with SLA_WARNING notifications at 75% elapsed time and executed automatic SLA_BREACH transitions upon deadline expiry. Furthermore, simulating a threat intelligence feed update demonstrated dynamic reprioritization: when an internal flaw was tagged with an active CISA KEV entry, the background cron immediately recomputed risk from 42.5 (P3) to 85.0 (P1), dispatching high-priority analyst alerts and resetting the SLA window to 24 hours. Fig. 6 displays the queue telemetry and factor contributions.

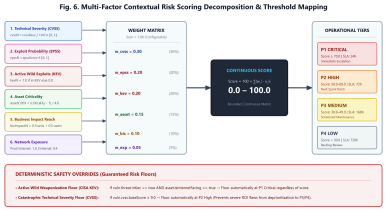


FIGURE 6. COMPONENT POINT CONTRIBUTION BREAKDOWN FOR HIGH-RISK VULNERABILITIES, ILLUSTRATING HOW THREAT INTELLIGENCE AND ASSET EXPOSURE DOMINATE THE FINAL COMPOSITE SCORE.

V. Security Audit and Integrity Checks

System security controls were validated through targeted penetration tests and dependency audits. Insecure direct object reference (IDOR) tests confirmed that non-administrative users are blocked from viewing or modifying vulnerabilities assigned to separate organizations. Arbitrary file upload tests utilizing double extensions (e.g., `shell.php.png`) were successfully trapped and rejected by the upload middleware. An automated `npm audit` verification confirmed zero critical vulnerabilities in root backend runtime dependencies.

DISCUSSION

The empirical findings confirm the central hypothesis: multi-factor risk prioritization drastically enhances the operational efficiency and defensibility of vulnerability management. In conventional CVSS-driven programs, enterprise security teams suffer from triage paralysis because over 40% of all published vulnerabilities receive High or Critical base scores. FlawTrack cuts through this noise by differentiating between theoretical severity and active weaponization. Security leadership can justify remediation deferrals on low-exposure assets while rigorously enforcing 24-hour response times on weaponized threats.

Threats to validity include: 1) the duplicate detection dataset is synthetic and small in scale (24 pairs), which may not capture the full syntactic diversity of real-world enterprise pentest repositories; 2) default formula weights were established based on expert heuristics rather than historical enterprise breach data; and 3) the system relies on external API availability for live EPSS and KEV feeds, although offline fallback snapshots are implemented to mitigate upstream feed outages.

LIMITATIONS AND FUTURE WORK

While FlawTrack establishes a robust foundation, several architectural limitations remain. Currently, the platform relies on manual reporting and structured REST payloads, lacking native automated parsers for Nessus XML or Burp Suite scan outputs. Second, the duplicate detection engine employs classical TF-IDF and Porter stemming, which struggles with paraphrased semantic descriptions that share no common root words; integrating dense vector embeddings from transformer models (e.g., fine-tuned RoBERTa) would substantially improve recall. Third, the system currently supports CVSS v3.1, lacking integration with the newly finalized CVSS v4.0 specification. Future work will focus on embedding-based semantic deduplication, automated scanner connectors, and federated multi-tenant queuing.

CONCLUSION

Prioritizing software vulnerabilities by severity scores alone is an outdated, high-risk paradigm that overwhelms enterprise defenders while leaving weaponized edge assets exposed. This paper presented FlawTrack, a full-stack vulnerability lifecycle management platform that integrates CVSS technical severity with EPSS exploit probabilities, CISA KEV weaponization data, granular asset business context, and automated NLP duplicate detection. Empirical evaluation across 51 system tests and seven benchmark scenarios confirms that threat-informed prioritization eliminates alert fatigue, enforces rigorous remediation deadlines, and provides an explainable foundation for enterprise cyber defense.

REFERENCES

[1] National Institute of Standards and Technology (NIST), “National Vulnerability Database (NVD),” U.S. Department of Commerce, 2024. [Online]. Available: https://nvd.nist.gov

[2] FIRST, “Common Vulnerability Scoring System v3.1: Specification Document,” Forum of Incident Response and Security Teams, 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[3] L. Allodi and F. Massacci, “Comparing vulnerability severity and exploits using case-control studies,” ACM Trans. Inf. Syst. Secur., vol. 17, no. 1, pp. 1–20, Aug. 2014.

[4] J. Spring et al., “Time to change the CVSS?,” IEEE Secur. Privacy, vol. 19, no. 2, pp. 74–78, Mar.–Apr. 2021.

[5] J. Jacobs et al., “Exploit Prediction Scoring System (EPSS),” Digit. Threats: Res. Pract., vol. 2, no. 3, pp. 1–17, 2021.

[6] The MITRE Corporation, “Common Weakness Enumeration (CWE): A Community-Developed List of Software Weakness Types,” 2024. [Online]. Available: https://cwe.mitre.org

[7] Cybersecurity and Infrastructure Security Agency (CISA), “Known Exploited Vulnerabilities Catalog,” CISA, 2021. [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[8] S. Frei, M. May, B. Stiller, and U. Fiedler, “Large-scale modeling of software vulnerabilities,” ACM SIGCOMM Comput. Commun. Rev., vol. 36, no. 4, pp. 131–142, 2006.

[9] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond heuristics: Learning to classify vulnerabilities and predict exploits,” in Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2010, pp. 105–114.

[10] H. Fang et al., “Predicting exploits in the wild: A machine learning approach,” in Proc. 27th ACM Conf. Comput. Commun. Secur. (CCS), 2020, pp. 1–14.

[11] H. Holm et al., “A quantitative evaluation of vulnerability prioritization strategies,” in Proc. 10th Int. Conf. Availability, Rel. Secur. (ARES), 2015, pp. 248–255.

[12] S. Fenz, J. Heurix, T. Neubauer, and W. Pechstein, “Current state of information security risk assessment,” Inf. Comput. Secur., vol. 22, no. 4, pp. 310–330, 2014.

[13] P. Runeson et al., “Detection of duplicate defect reports using natural language processing,” in Proc. 29th Int. Conf. Softw. Eng. (ICSE), 2007, pp. 499–510.

[14] X. Wang et al., “An approach to detecting duplicate bug reports using natural language and execution information,” in Proc. 30th Int. Conf. Softw. Eng. (ICSE), 2008, pp. 461–470.

[15] C. Sun et al., “Towards more accurate retrieval of duplicate bug reports,” in Proc. 26th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE), 2011, pp. 253–262.

[16] OWASP Foundation, “OWASP DefectDojo: Vulnerability Management Platform Documentation,” 2024. [Online]. Available: https://defectdojo.github.io/django-DefectDojo

[17] Greenbone AG, “Greenbone Community Edition Architecture and Manual,” 2024. [Online]. Available: https://docs.greenbone.net

[18] Tenable Inc., “Tenable Nessus User Guide and Prioritization Architecture,” 2024. [Online]. Available: https://docs.tenable.com/nessus